NAME: ANKIT KUMAR

ENROLLMENT: S25MCAG0010

LAB ASSIGNMENT-5

1. Find the average marks of each course but show only courses whose average is ≥ 75.

```
mysql> SELECT c.course_name, AVG(e.marks) AS avg_marks
    -> FROM Enrollment e
    -> JOIN Course c ON e.course_id = c.course_id
    -> GROUP BY c.course_name
    -> HAVING AVG(e.marks) >= 75;
+------------------+-----------+
| course_name      | avg_marks |
+------------------+-----------+
| DBMS             |   76.8333 |
| AI               |   80.0000 |
| Data Structures  |   77.3333 |
| Machine Learning |   92.0000 |
+------------------+-----------+
4 rows in set (0.02 sec)
```

2. Count how many students enrolled in each instructor's courses.

```
mysql> SELECT c.instructor, COUNT(e.student_id) AS total_students
    -> FROM course c
    -> JOIN enrollment e ON c.course_id=e.course_id
    -> GROUP BY c.instructor;
+------------+----------------+
| instructor | total_students |
+------------+----------------+
| Dr. Rao    |              8 |
| Dr. Sharma |              5 |
| Dr. Mehta  |              3 |
+------------+----------------+
3 rows in set (0.01 sec)
```

3. Display department-wise maximum marks obtained by any student.

```
mysql> SELECT s.department, MAX(e.marks) AS max_marks
    -> FROM student s
    -> JOIN enrollment e ON s.student_id=e.student_id
    -> GROUP BY s.department;
+------------+-----------+
| department | max_marks |
+------------+-----------+
| CSE        |        92 |
| IT         |        82 |
| ECE        |        74 |
+------------+-----------+
3 rows in set (0.01 sec)
```

4. Find the course with the highest average marks.

```
mysql> SELECT c.course_name, AVG(e.marks) AS avg_marks
    -> FROM course c
    -> JOIN enrollment e ON c.course_id=e.course_id
    -> GROUP BY c.course_name
    -> ORDER BY avg_marks DESC
    -> LIMIT 1;
+-----------------+-----------+
| course_name     | avg_marks |
+-----------------+-----------+
| Machine Learning |   92.0000 |
+-----------------+-----------+
1 row in set (0.01 sec)
```

5. List departments where more than 2 students have appeared in DBMS.

```
mysql> SELECT s.department, COUNT(*) AS total_students
    -> FROM Student s
    -> JOIN Enrollment e ON s.student_id = e.student_id
    -> JOIN Course c ON e.course_id = c.course_id
    -> WHERE c.course_name = 'DBMS'
    -> GROUP BY s.department
    -> HAVING COUNT(*) > 2;
+------------+---------------+
| department | total_students |
+------------+---------------+
| CSE        |             3 |
+------------+---------------+
1 row in set (0.01 sec)
```

6. Find instructors whose average student marks exceed the overall average marks of all students.

```
mysql> SELECT c.instructor
    -> FROM Course c
    -> JOIN Enrollment e ON c.course_id = e.course_id
    -> GROUP BY c.instructor
    -> HAVING AVG(e.marks) >
    -> (SELECT AVG(marks) FROM Enrollment);
+------------+
| instructor |
+------------+
| Dr. Sharma |
+------------+
1 row in set (0.04 sec)
```

7. Show courses where minimum marks ≥ 60.

```
mysql> SELECT c.course_name
    -> FROM Course c
    -> JOIN Enrollment e ON c.course_id = e.course_id
    -> GROUP BY c.course_name
    -> HAVING MIN(e.marks) >= 60;
+------------------+
| course_name      |
+------------------+
| DBMS             |
| AI               |
| Data Structures  |
| Operating System |
| Machine Learning |
+------------------+
5 rows in set (0.02 sec)
```

8. Find students who scored more than the average marks of their respective course.

```
mysql> SELECT s.name, c.course_name, e.marks
    -> FROM Student s
    -> JOIN Enrollment e ON s.student_id = e.student_id
    -> JOIN Course c ON e.course_id = c.course_id
    -> WHERE e.marks >
    -> (
    -> SELECT AVG(marks)
    -> FROM Enrollment
    -> WHERE course_id = e.course_id
    -> );
+-------+------------------+-------+
| name  | course_name      | marks |
+-------+------------------+-------+
| Ankit | DBMS             |    85 |
| Amit  | DBMS             |    90 |
| Amit  | Data Structures  |    88 |
| Sneha | DBMS             |    80 |
| Sneha | Operating System |    70 |
| Priya | AI               |    82 |
| Neha  | AI               |    85 |
+-------+------------------+-------+
7 rows in set (0.01 sec)
```

9. List students who scored less than the average marks of DBMS.

```
mysql> SELECT s.name, e.marks
    -> FROM Student s
    -> JOIN Enrollment e ON s.student_id = e.student_id
    -> JOIN Course c ON e.course_id = c.course_id
    -> WHERE c.course_name='DBMS'
    -> AND e.marks <
    -> (
    -> SELECT AVG(e2.marks)
    -> FROM Enrollment e2
    -> JOIN Course c2 ON e2.course_id = c2.course_id
    -> WHERE c2.course_name='DBMS'
    -> );
+-------+-------+
| name  | marks |
+-------+-------+
| Rahul |    72 |
| Rohan |    60 |
| Karan |    74 |
+-------+-------+
3 rows in set (0.01 sec)
```

10. Find courses whose average marks are higher than AI’s average marks.

```
mysql> SELECT c.course_name
    -> FROM Course c
    -> JOIN Enrollment e ON c.course_id = e.course_id
    -> GROUP BY c.course_name
    -> HAVING AVG(e.marks) >
    -> (
    -> SELECT AVG(e2.marks)
    -> FROM Enrollment e2
    -> JOIN Course c2 ON e2.course_id = c2.course_id
    -> WHERE c2.course_name='AI'
    -> );
+------------------+
| course_name      |
+------------------+
| Machine Learning |
+------------------+
1 row in set (0.01 sec)
```

11. Display students who are enrolled in more courses than the average enrollment per student.

```
mysql> SELECT student_id
    -> FROM Enrollment
    -> GROUP BY student_id
    -> HAVING COUNT(course_id) >
    -> (
    -> SELECT AVG(course_count)
    -> FROM
    -> (
    -> SELECT COUNT(course_id) AS course_count
    -> FROM Enrollment
    -> GROUP BY student_id
    -> ) AS avg_table
    -> );
+------------+
| student_id |
+------------+
|          1 |
|          2 |
|          5 |
+------------+
3 rows in set (0.02 sec)
```

12. Find names of students who have never scored below 70 in any course.

```
mysql> SELECT s.name
    -> FROM Student s
    -> JOIN Enrollment e ON s.student_id = e.student_id
    -> GROUP BY s.name
    -> HAVING MIN(e.marks) >= 70;
+----------+
| name     |
+----------+
| Himanshu |
| Ankit    |
| Jayant   |
| Priya    |
| Ritu     |
+----------+
5 rows in set (0.01 sec)
```

13. List students who are enrolled in all courses taught by Dr. Sharma.

```
mysql> SELECT s.name
    -> FROM Student s
    -> JOIN Enrollment e ON s.student_id = e.student_id
    -> JOIN Course c ON e.course_id = c.course_id
    -> WHERE c.instructor = 'Dr. Sharma'
    -> GROUP BY s.student_id
    -> HAVING COUNT(DISTINCT c.course_id) =
    -> (
    -> SELECT COUNT(*)
    -> FROM Course
    -> WHERE instructor='Dr. Sharma'
    -> );
+------+
| name |
+------+
| Ritu |
+------+
1 row in set (0.01 sec)
```

14. Find instructors who teach more than one course.

```
mysql> SELECT instructor
    -> FROM Course
    -> GROUP BY instructor
    -> HAVING COUNT(course_id) > 1;
+------------+
| instructor |
+------------+
| Dr. Chanda |
+------------+
1 row in set (0.00 sec)
```

15. Display top 3 students based on average marks.

```
mysql> SELECT s.name, AVG(e.marks) AS avg_marks
    -> FROM Student s
    -> JOIN Enrollment e ON s.student_id = e.student_id
    -> GROUP BY s.name
    -> ORDER BY avg_marks DESC
    -> LIMIT 3;
+----------+-----------+
| name     | avg_marks |
+----------+-----------+
| Jayant   |   90.0000 |
| Himanshu |   82.5000 |
| Priya    |   81.5000 |
+----------+-----------+
3 rows in set (0.00 sec)
```

16. Find the second highest mark obtained in DBMS.

```
mysql> SELECT DISTINCT marks
    -> FROM Enrollment e
    -> JOIN Course c ON e.course_id = c.course_id
    -> WHERE c.course_name='DBMS'
    -> ORDER BY marks DESC
    -> LIMIT 1 OFFSET 1;
+-------+
| marks |
+-------+
|    75 |
+-------+
1 row in set (0.00 sec)
```

17. Show top 2 courses with the highest number of enrollments.

```
mysql> SELECT c.course_name, COUNT(e.student_id) AS total_students
    -> FROM Course c
    -> JOIN Enrollment e ON c.course_id = e.course_id
    -> GROUP BY c.course_name
    -> ORDER BY total_students DESC
    -> LIMIT 2;
+-------------+----------------+
| course_name | total_students |
+-------------+----------------+
| DBMS        |              5 |
| IMDAI       |              2 |
+-------------+----------------+
2 rows in set (0.00 sec)
```

18. Display output as
    Student: Amit (CSE) - Avg Marks: 80.00

```
mysql> SELECT CONCAT('Student: ', s.name, ' (', s.department, ') - Avg Marks: ',
    -> ROUND(AVG(e.marks),2)) AS result
    -> FROM Student s
    -> JOIN Enrollment e ON s.student_id = e.student_id
    -> GROUP BY s.student_id;
+-------------------------------------------+
| result                                    |
+-------------------------------------------+
| Student: Himanshu (CSE) - Avg Marks: 82.50 |
| Student: Ankit (CSE) - Avg Marks: 72.50    |
| Student: Shyam (IT) - Avg Marks: 60.00     |
| Student: Jayant (CSE) - Avg Marks: 90.00   |
| Student: Priya (IT) - Avg Marks: 81.50     |
| Student: Pawash (IT) - Avg Marks: 69.00    |
| Student: Ritu (CSE) - Avg Marks: 78.00     |
+-------------------------------------------+
7 rows in set (0.01 sec)
```

19. Show
    Course: DBMS | Instructor: Dr. Rao | Avg Marks: XX

```
mysql> SELECT CONCAT('Course: ', c.course_name,
    -> ' | Instructor: ', c.instructor,
    -> ' | Avg Marks: ', ROUND(AVG(e.marks),2)) AS result
    -> FROM Course c
    -> JOIN Enrollment e ON c.course_id = e.course_id
    -> GROUP BY c.course_id;
+-----------------------------------------------------------+
| result                                                    |
+-----------------------------------------------------------+
| Course: DBMS | Instructor: Dr. Chanda | Avg Marks: 71.80  |
| Course: IMDAI | Instructor: Dr. Rawat | Avg Marks: 77.50  |
| Course: IR | Instructor: Dr. Solanki | Avg Marks: 90.00   |
| Course: Networks | Instructor: Dr. Rao | Avg Marks: 88.00 |
| Course: AI | Instructor: Dr. Sharma | Avg Marks: 78.00    |
+-----------------------------------------------------------+
5 rows in set (0.00 sec)
```

20. Find students who scored above department average but below overall university average.

```
mysql> SELECT s.name
    -> FROM Student s
    -> JOIN Enrollment e ON s.student_id = e.student_id
    -> WHERE e.marks >
    -> (
    -> SELECT AVG(e2.marks)
    -> FROM Student s2
    -> JOIN Enrollment e2 ON s2.student_id = e2.student_id
    -> WHERE s2.department = s.department
    -> )
    -> AND e.marks <
    -> (
    -> SELECT AVG(marks) FROM Enrollment
    -> );
+-------+
| name  |
+-------+
| Priya |
+-------+
1 row in set (0.01 sec)
```